Development environments

Software Management & Development · Session 1



IE University · MFT & MCSDI

This course

What you will do

- Build **one application** end to end: Python backend, database, REST API, web frontend
- Test it, package it in containers, deploy it to **Azure**
- Work like a software team: Git, code review, small deliveries

How

- Short concepts, then a **lab** every session
- A **group project** that follows the same path

The running example: **ledger**

A personal-finance app: accounts, transactions, balances, spending by category.

S3–S6	Python project on your laptop
S7–S10	+ PostgreSQL, API, tests, frontend
S11–S12	all in Docker Compose
S13–S14	the same images on Azure

Evaluation and AI

Final exam	30 %
Group project + live demo	40 %
Individual work (labs, homework)	20 %
Participation	10 %

Generative AI: encouraged

- You are **responsible** for every line you submit
- **Acknowledge** which tool, for what, which prompts
- In the exam and the demo you **explain** your code

Why environments matter

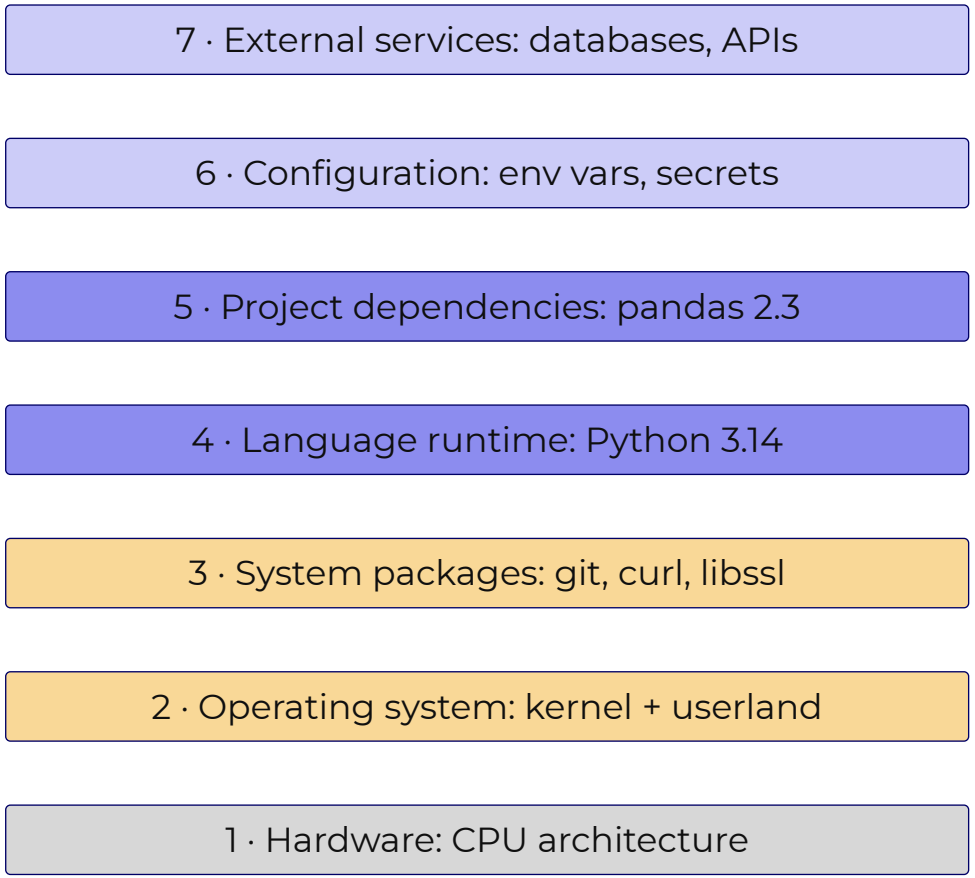
“It works on my machine”

Ana’s fraud-detection script works on her laptop. On the server:

1. `ModuleNotFoundError: No module named 'pandas'` — *dependencies*
2. `SyntaxError` on a `match` statement (Python 3.9 vs 3.12) — *runtime*
3. `C:\Users\ana\Desktop\cards.csv` not found on Linux — *operating system*
4. Runs fine… but flags almost nothing: `RISK_THRESHOLD` was set only on Ana’s laptop — *configuration*

> None of these is a bug in Ana’s logic. The last one is the dangerous one: it **seems** to work.

The layers of an environment



Everything **except your code** that decides how it runs.

A difference in **any** layer can break it.

The goal: the environment is **the same everywhere**, or at least **known**: every laptop, the test server, production.

Reproducibility

An environment is **reproducible** if anyone can rebuild it **from files** and get the same result.

1. Document

README: “install Python 3.14, then…”

People skip steps; it goes out of date.

2. Automate

`pyproject.toml` + `uv.lock`, `.env.example`

Fixes runtime and dependencies.

3. Package

VM image, **container image**

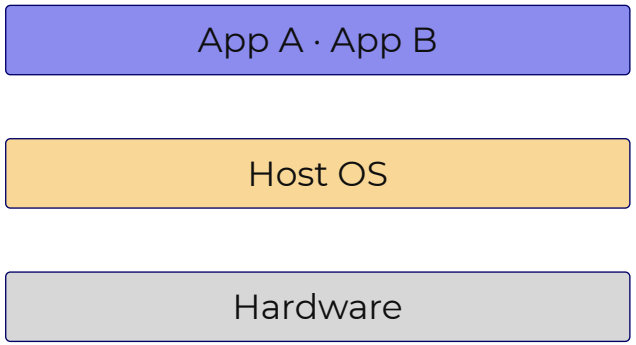
Ships OS + runtime + dependencies with the code.

> If it isn’t in a file in the repository, it isn’t reproducible.

Host, virtual machine, container

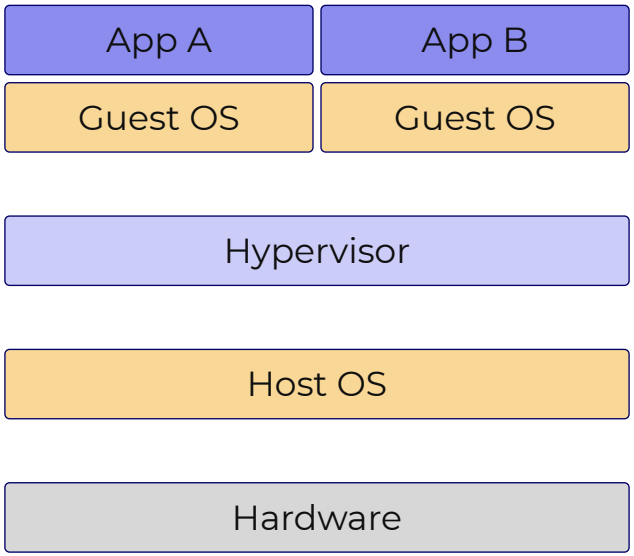
Three ways to run software

Host



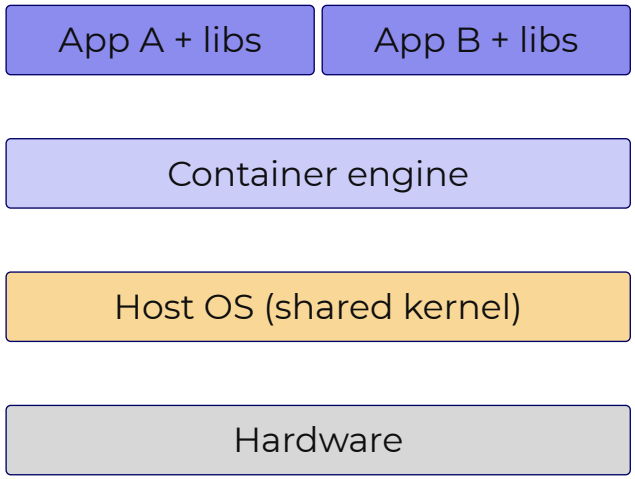
Everything shared: upgrades for one project break another.

Virtual machines



Each VM is a full computer with its **own kernel**.

Containers



Isolated processes that **share the kernel**.

VM or container?

	Virtual machine	Container
Isolates	the whole computer	processes, files, network
Own kernel	yes	no: shares the host's
Starts in	tens of seconds	under a second
Size	gigabytes	megabytes to hundreds of MB
Different OS?	yes (Linux on a Mac…)	same kernel type only
Use it for	a full separate machine	packaging and shipping one app

Docker Desktop on macOS and Windows runs a **small Linux VM** in the background. Your containers live inside it.

The processor matters too

Two architectures

- `x86_64` = `amd64`: most Windows PCs and cloud servers
- `arm64` = `aarch64`: Apple Silicon Macs, Snapdragon PCs, some cloud servers

Compiled code runs only on its architecture. Python is portable, but NumPy, database drivers, cryptography… are not.

The classic cloud failure

Build an image on an M-series Mac → push to an `x86_64` server →

```
exec format error
```

Fix: build for the target **platform** (`linux/amd64`). We do it in session 13.

From your laptop to the cloud

Who manages what

Model	You manage	The provider manages	Azure example
On-premises	everything	nothing	—
IaaS	OS, runtime, deps, app, data	hardware, network, virtualization	Virtual Machines
Serverless containers	your image + its configuration	everything below + scaling	Container Apps
Managed services	your data and settings	the whole service	PostgreSQL, Static Web Apps
SaaS	how you use it	everything	Microsoft 365, GitHub

The **container image** is yours in every model, and the unit that travels from laptop to cloud unchanged.

The course map

S1–S2 · a known, reproducible environment

S3–S6 · Python project, uv, Git, structure

S7–S10 · PostgreSQL in Docker, FastAPI, tests, frontend

S11–S12 · everything in Docker Compose

S13–S14 · the same images on Azure

The cloud: Azure

- The syllabus uses Google Cloud names: Artifact Registry → **Container Registry**, Cloud Run → **Container Apps**, App Engine → **Static Web Apps**

Lab 1: know your machine

Today's lab

1. Open a terminal: `whoami`, `pwd`, `ls`
2. OS and architecture: `uname -m`, `sw_vers`…
3. Memory, disk, virtualization
4. Can you install software?
5. Create your **GitHub** account
6. Fill in your **machine card**

> Nothing gets installed today. Everything only **reads** information.

Bring the card to **session 2**, where we set up the course environment:

- native tools + Docker Desktop,
- an Ubuntu VM,
- or a cloud dev environment.